TÉCNICO LISBOA

Software Testing and Validation

2026

Exercise 4

Chosen tool: **Valgrind**

Group - 12

Valgrind Memcheck is a dynamic analysis tool that detects memory-management errors by executing a program and monitoring its memory operations at runtime. Unlike the static-analysis tools used in Exercise 1 (Infer and clang-analyzer), Valgrind does not inspect source code to predict possible bugs; instead, it reports errors that actually occur during a concrete execution. It also differs from AFL++ in Exercise 2, which automatically generates inputs to explore program behaviour, and from KLEE in Exercise 3, which symbolically reasons about many possible execution paths. Valgrind only analyses the execution paths triggered by the provided inputs.

For this exercise, hashmap.c was extended with a custom driver to exercise additional buggy paths. The modified test uses long keys to trigger the unsafe malloc(sizeof(key)) + strcpy sequence, inserts multiple keys to force bucket growth, and performs repeated hashmap_get calls without freeing the returned memory. Designing these inputs required prior knowledge of the bugs found in Exercise 1 — a known limitation of dynamic analysis, since Valgrind cannot discover bugs that are never triggered. These targeted tests were intentionally constructed to provide more extensive coverage of the known defects. Valgrind is particularly effective at detecting memory leaks, invalid memory accesses, use-after-free errors, and double frees.

```
gcc -g -o hashmap_test hashmap.c

gcc -g -DDRIVER_MAIN hashmap.c hashmap_driver.c -o hashmap_driver
```

```
root@6f717a90affe:/project/2-Ex4# gcc -g -o hashmap_test hashmap.c
root@6f717a90affe:/project/2-Ex4# gcc -g -DDRIVER_MAIN hashmap.c hashmap_driver.c -o hashmap_driver
root@6f717a90affe:/project/2-Ex4# ls
hashmap.c  hashmap.h  hashmap_driver  hashmap_driver.c  hashmap_driver.dSYM  hashmap_test  hashmap_test.dSYM
root@6f717a90affe:/project/2-Ex4#
```

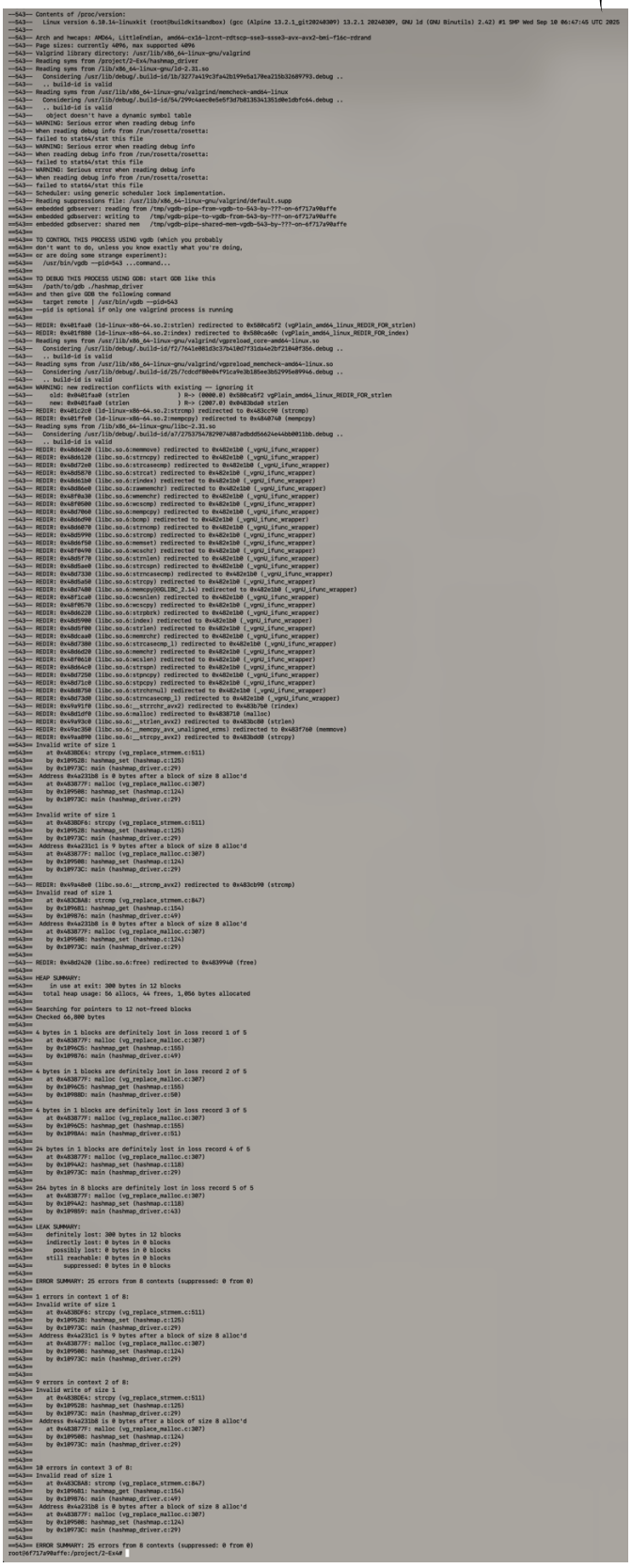
```
root@6f717a90affe:/project/2-Ex4# valgrind --leak-check=full --track-origins=yes ./hashmap_driver
==539== Memcheck, a memory error detector
==539== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==539== Using Valgrind-3.16.1 and LibVEX; rerun with -h for copyright info
==539== Command: ./hashmap_driver
==539==
==539== Invalid write of size 1
==539==    at 0x483BDE4: strcpy (vg_replace_strmem.c:511)
==539==    by 0x109528: hashmap_set (hashmap.c:125)
==539==    by 0x10973C: main (hashmap_driver.c:29)
==539==  Address 0x4a231b8 is 0 bytes after a block of size 8 alloc'd
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x109508: hashmap_set (hashmap.c:124)
==539==    by 0x10973C: main (hashmap_driver.c:29)
==539==
==539== Invalid write of size 1
==539==    at 0x483BDF6: strcpy (vg_replace_strmem.c:511)
==539==    by 0x109528: hashmap_set (hashmap.c:125)
==539==    by 0x10973C: main (hashmap_driver.c:29)
==539==  Address 0x4a231c1 is 9 bytes after a block of size 8 alloc'd
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x109508: hashmap_set (hashmap.c:124)
==539==    by 0x10973C: main (hashmap_driver.c:29)
==539==
==539== Invalid read of size 1
==539==    at 0x483CBA8: strcmp (vg_replace_strmem.c:847)
==539==    by 0x1096B1: hashmap_get (hashmap.c:154)
==539==    by 0x109876: main (hashmap_driver.c:49)
==539==  Address 0x4a231b8 is 0 bytes after a block of size 8 alloc'd
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x109508: hashmap_set (hashmap.c:124)
==539==    by 0x10973C: main (hashmap_driver.c:29)
==539==
==539==
==539== HEAP SUMMARY:
==539==     in use at exit: 300 bytes in 12 blocks
==539==   total heap usage: 56 allocs, 44 frees, 1,056 bytes allocated
==539==
==539== 4 bytes in 1 blocks are definitely lost in loss record 1 of 5
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x1096C5: hashmap_get (hashmap.c:155)
==539==    by 0x109876: main (hashmap_driver.c:49)
==539==
==539== 4 bytes in 1 blocks are definitely lost in loss record 2 of 5
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x1096C5: hashmap_get (hashmap.c:155)
==539==    by 0x10988D: main (hashmap_driver.c:50)
==539==
==539== 4 bytes in 1 blocks are definitely lost in loss record 3 of 5
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x1096C5: hashmap_get (hashmap.c:155)
==539==    by 0x1098A4: main (hashmap_driver.c:51)
==539==
==539== 24 bytes in 1 blocks are definitely lost in loss record 4 of 5
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x1094A2: hashmap_set (hashmap.c:118)
==539==    by 0x10973C: main (hashmap_driver.c:29)
==539==
==539== 264 bytes in 8 blocks are definitely lost in loss record 5 of 5
==539==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==539==    by 0x1094A2: hashmap_set (hashmap.c:118)
==539==    by 0x109859: main (hashmap_driver.c:43)
==539==
==539== LEAK SUMMARY:
==539==    definitely lost: 300 bytes in 12 blocks
==539==    indirectly lost: 0 bytes in 0 blocks
==539==      possibly lost: 0 bytes in 0 blocks
==539==    still reachable: 0 bytes in 0 blocks
==539==         suppressed: 0 bytes in 0 blocks
==539==
==539== For lists of detected and suppressed errors, rerun with: -s
==539== ERROR SUMMARY: 25 errors from 8 contexts (suppressed: 0 from 0)
root@6f717a90affe:/project/2-Ex4#
```

Three distinct bug classes found:

- `Invalid write of size 1` at `hashmap.c:125` — `strcpy` overflows the 8-byte buffer allocated by `malloc(sizeof(key))` at line 124. Valgrind catches both the first byte past the boundary and the null terminator 9 bytes past.
- `Invalid read of size 1` at `hashmap.c:154` — `strcmp` in `hashmap_get` later reads into the same overflowed region.
- 12 blocks definitely lost (300 bytes total):
- 3 × 4 bytes from `hashmap_get` return values never freed (lines 49–51 of driver)
- 1 × 24 bytes + 8 × 33 bytes from `hashmap_set` at line 118 — old `field->entries` pointers leaked on bucket growth

```
root@6f717a90affe:/project/2-Ex4# valgrind --leak-check=full --track-origins=yes ./hashmap_test
==541== Memcheck, a memory error detector
==541== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==541== Using Valgrind-3.16.1 and LibVEX; rerun with -h for copyright info
==541== Command: ./hashmap_test
==541==
==541==
==541== HEAP SUMMARY:
==541==     in use at exit: 4 bytes in 1 blocks
==541==   total heap usage: 6 allocs, 5 frees, 184 bytes allocated
==541==
==541== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==541==    at 0x483877F: malloc (vg_replace_malloc.c:307)
==541==    by 0x1096D5: hashmap_get (hashmap.c:155)
==541==    by 0x109764: main (hashmap.c:170)
==541==
==541== LEAK SUMMARY:
==541==    definitely lost: 4 bytes in 1 blocks
==541==    indirectly lost: 0 bytes in 0 blocks
==541==      possibly lost: 0 bytes in 0 blocks
==541==    still reachable: 0 bytes in 0 blocks
==541==         suppressed: 0 bytes in 0 blocks
==541==
==541== For lists of detected and suppressed errors, rerun with: -s
==541== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
root@6f717a90affe:/project/2-Ex4#
```

Only 1 error, 1 block (4 bytes) lost — the minimal test never calls `hashmap_set` with a long key or causes bucket collisions, so Valgrind sees almost nothing.



```
==543== Invalid write of size 1

==543==    at 0x483BDE4: strcpy
(vg_replace_strmem.c:511)

==543==    by 0x109528: hashmap_set (hashmap.c:125)

==543==    by 0x10973C: main (hashmap_driver.c:29)

==543==  Address 0x4a231b8 is 0 bytes after a block of
size 8 alloc'd

==543==    at 0x483877F: malloc
(vg_replace_malloc.c:307)

==543==    by 0x109508: hashmap_set (hashmap.c:124)

==543== Invalid read of size 1

==543==    at 0x483CBA8: strcmp
(vg_replace_strmem.c:847)

==543==    by 0x1096B1: hashmap_get (hashmap.c:154)

==543==    by 0x109876: main (hashmap_driver.c:49)

==543== 4 bytes in 1 blocks are definitely lost

==543==    at 0x483877F: malloc
(vg_replace_malloc.c:307)

==543==    by 0x1096C5: hashmap_get (hashmap.c:155)

==543== 24 bytes in 1 blocks are definitely lost

==543==    at 0x483877F: malloc
(vg_replace_malloc.c:307)

==543==    by 0x1094A2: hashmap_set (hashmap.c:118)

==543== 264 bytes in 8 blocks are definitely lost

==543==    at 0x483877F: malloc
(vg_replace_malloc.c:307)

==543==    by 0x1094A2: hashmap_set (hashmap.c:118)

==543== LEAK SUMMARY:

==543==    definitely lost: 300 bytes in 12 blocks

==543== ERROR SUMMARY: 25 errors from 8 contexts
```

Same findings as Screenshot 1 but the `--543--` REDIR lines show Valgrind intercepting every memory function (`malloc`, `free`, `strcpy`, `strcmp`, `memcpy`) at runtime — confirming it is a true dynamic analysis tool that observes actual execution rather than reasoning about code statically.

TÉCNICO LISBOA

Concluding statement

Valgrind with a targeted driver confirmed the buffer overflow at line 125 and the bucket-growth memory leak at line 127 — both also flagged by Infer and Clang — but missed all null-dereference bugs (lines 61, 63, 118, 131, 155) since those require malloc to return NULL, a path no concrete execution can reliably force.